CSS3 - переходы
CSS3 - анимация

CSS3-переходы

CSS3-переходы позволяют анимировать исходное значение CSS-свойства на новое значение с течением времени, управляя скоростью смены значений свойств. Задается через свойство `transition`

CSS переходы позволяют вам решить какие свойства нужно анимировать (перечислением их явно), когда анимация начнётся (установкой задержки), как долго переход будет выполняться (установкой продолжительности), а также как она будет запущена (определением функции по времени, например линейно или быстро в начале, медленно в конце).

Смена свойств происходит при наступлении определенного события, которое описывается соответствующим псевдоклассом. Чаще всего используется псевдокласс :hover.

Переходы применяются ко всем элементам, а также к псевдоэлементам :before и :after.

transition-property Указывает имя или имена свойств, чьи переходы должны анимироваться.

```
transition-property: none | all | <свойство> [,<свойство>]*
```

```
.menu {
    position: absolute;
    height: 100px; width: 100%;
    background: #fc0;
    top: -100px;
    transition-property: top, background;
   }
   .menu:hover { top: 0; background: #fff;}
```

transition-duration Задаёт промежуток времени в секундах или миллисекундах, в течение которого должен осуществляться переход

```
transition-duration: <время> [,<время>]*
```

```
.menu {
    position: absolute;
    height: 100px; width: 100%;
    background: #fc0;
    top: -100px;
    transition-property: top, background;
    transition-duration: 0.2s;
   }
   .menu:hover { top: 0; background: #fff;}
```

Если разные свойства имеют разные значения для перехода, они указываются через запятую.

transition-delay устанавливает время ожидания перед запуском эффекта перехода

```
transition-delay: <время> [,<время>]*
```

```
.menu {
    position: absolute;
    height: 100px; width: 100%;
    background: #fc0;
    top: -100px;
    transition-property: top, background;
    transition-delay: 0.5s;
   }
   .menu:hover { top: 0; background: #fff;}
```

Если разные свойства имеют разные значения для перехода, они указываются через запятую.

transition-timing-function представляет собой математическую функцию, показывающую, как быстро по времени меняется указанное через transition-property значение свойства

transition-timing-function: ease | ease-in | ease-out | ease-in-out | linear | step-start | step-end | steps | cubic-bezier

ease Функция по умолчанию, переход начинается медленно, разгоняется быстро и замедляется в конце. Соответствует `cubic-bezier(0.25,0.1,0.25,1)`.

linear Переход происходит равномерно на протяжении всего времени, без колебаний в скорости. Соответствует `cubic-bezier(0,0,1,1)`.

ease-in Переход начинается медленно, а затем плавно ускоряется в конце. Соответствует `cubic-bezier(0.42,0,1,1`

ease-out Переход начинается быстро и плавно замедляется в конце. Соответствует `cubic-bezier(0,0,0.58,1)`.

ease-in-out Переход медленно начинается и медленно заканчивается. Соответствует `cubic-bezier(0.42,0,0.58,1)`.

cubic-bezier(x1, y1, x2, y2) Позволяет вручную установить значения от 0 до 1 для кривой ускорения. На этом сайте в сможете построить любую траекторию перехода.

transition-timing-function: ease | ease-in | ease-out | ease-in-out | linear | step-start | step-end | steps | cubic-bezier

steps`( n, <jumpterm>)` Отображает переход по *n остановкам вдоль перехода, отображая каждую остановку в течение* равных промежутков времени. Например, если *n* равно 5, есть 5 шагов. Независимо от того, выполняется ли перех временно на 0%, 20%, 40%, 60% и 80%, на 20%, 40%, 60%, 80% и 100%, или делает 5 остановок между 0% и 100% вдоль переход или выполнение 5 остановок, включая отметки 0% и 100% (на 0%, 25%, 50%, 75% и 100%), зависит от того, какой из следующих терминов перехода используется:

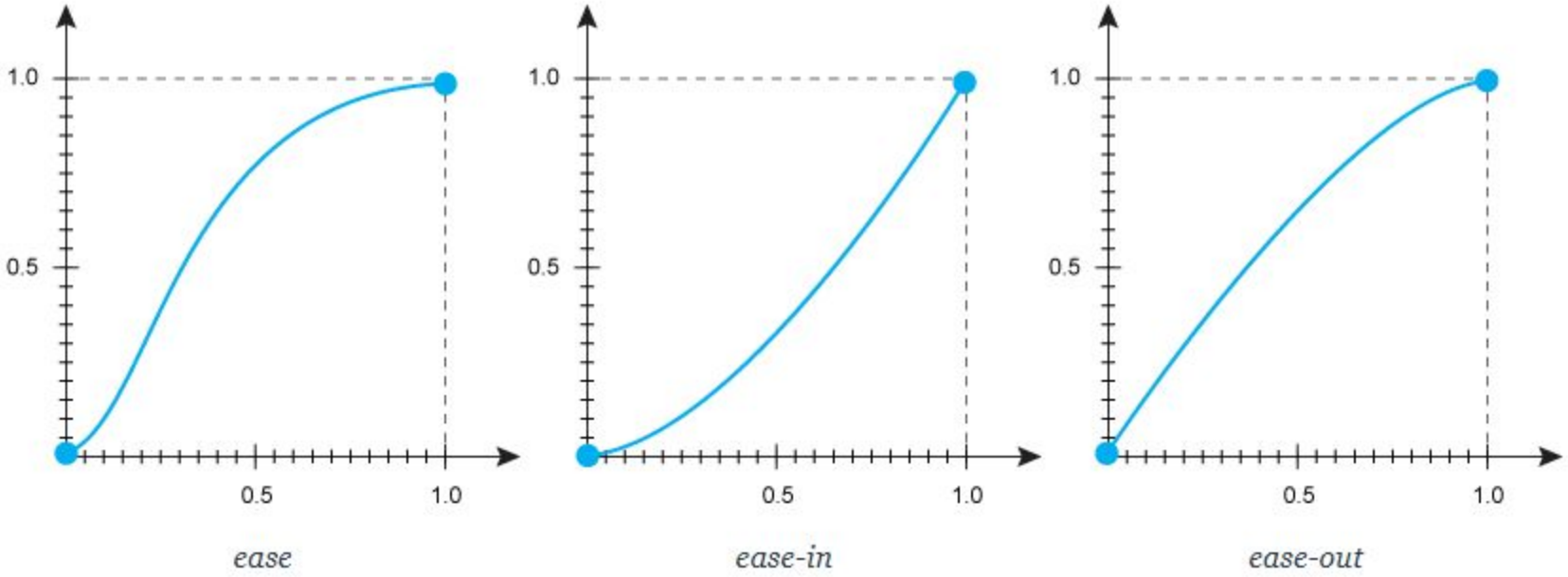
jump-start Обозначает непрерывную слева функцию, так что первый переход происходит, когда начинается переход;

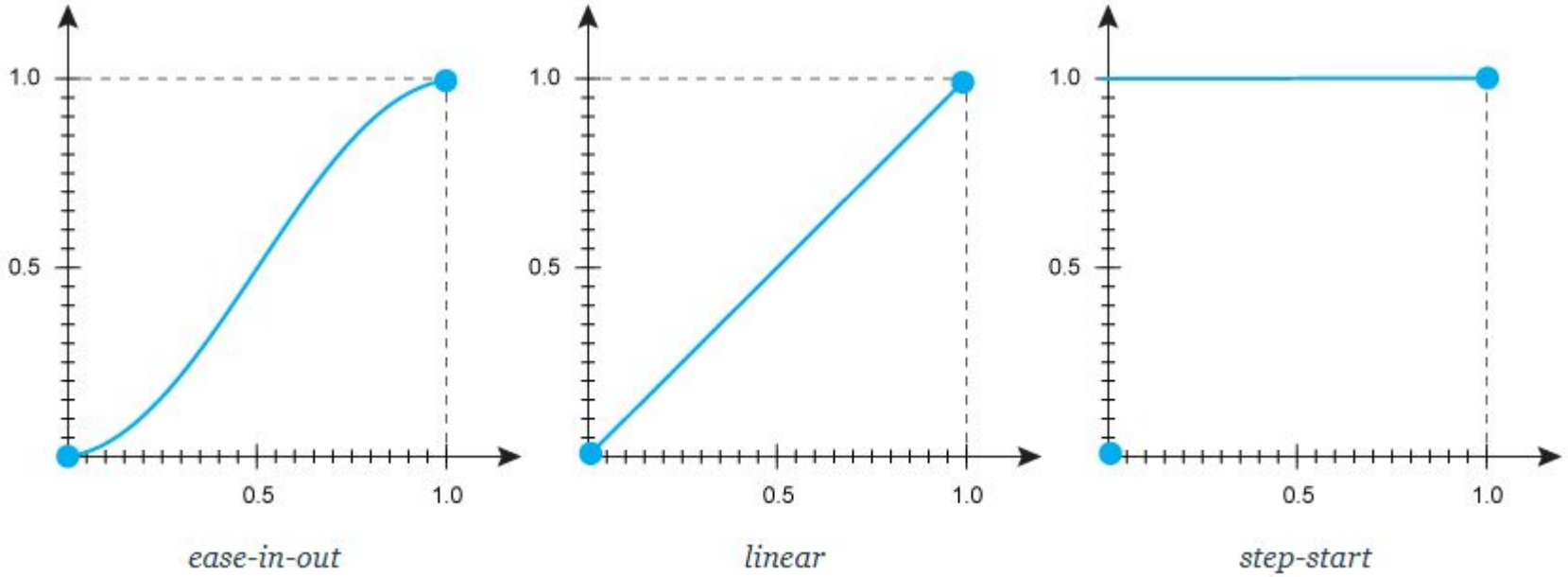
jump-end Обозначает непрерывную справа функцию, так что последний прыжок происходит по окончании анимации;

start Такой же как jump-start

end Такой же как jump-end.

transition-timing-function:





https://developer.mozilla.org/en-US/docs/Web/CSS/transition-timing-function

transition - краткая запись

```
transition: transition-property transition-duration
transition-timing-function transition-delay;
```

```
div {
transition-property: background, color;
transition-duration: 0.3s, 0.2s;
transition-timing-function: ease, linear;}
```

Кратко

```
div {transition: background 0.3s ease, color 0.2s linear;}
```

Пример1: изменение ширины

```
div {
  width: 100px;  height: 100px;
  background: red;
  transition: width 2s  linear;
}
div:hover {
  width: 300px;
}
```

```
<body><div></div></body>
```



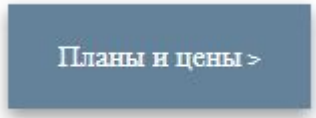


Пример2: изменение свойств заливки

```
a{
        display: inline-block;
         padding: 15px 25px;
         margin: 20px;
         background: #0067b8;
         box-shadow: 0 4px 8px 0 rgba(0,0,0,0);
        transition: all 2s ease-in-out;
    }
   a:hover{
         background: #64839b;
         box-shadow: 0 4px 8px 0 rgba(0,0,0,.4);
    }
```

```
<a href="#">  Планы и цены >  </a>
```





CSS3-трансформации

Трансформация — это изменение вида элемента, к которым относятся следующие визуальные модификации:

- **поворот,**
- **масштабирование,**
- **наклон и**
- **сдвиг**

Чтобы сделать трансформацию, к селектору добавляется свойство transform, а в качестве значения пишется функция трансформации и её параметры.

```
transform: <функция> [<функция>]* | none
```

`transform-origin:  50% 50%` - определяет исходную точку начала трансформации.

Трансформации преобразовывают элемент, не затрагивая остальные элементы веб-страницы, т.е. другие элементы не сдвигаются относительно него.
К элементам, которые могут быть трансформированы, относятся элементы с display: block; и display: inline-block

Существуют два вида CSS3-трансформаций – 2D и 3D

translate(x,y) - сдвиг

`translate(x,y)` Сдвигает элемент на новое место, перемещая относительно обычного положения вправо и вниз, используя координаты X и Y, не затрагивая при этом соседние элементы. Если нужно сдвинуть элемент влево или вверх, то нужно использовать отрицательные значения.

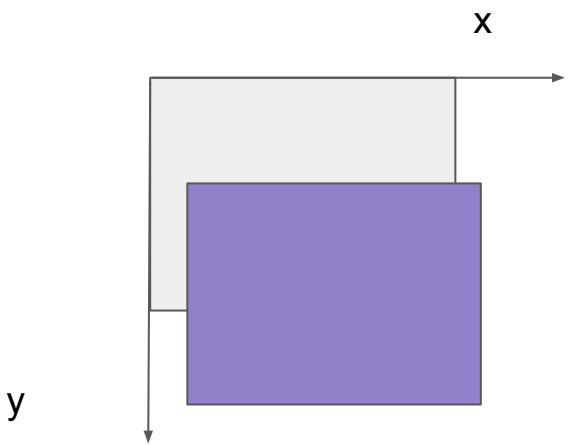
`translateX(n)` Сдвигает элемент относительно его обычного положения по оси X.
`translateY(n)` Сдвигает элемент относительно его обычного положения по оси Y.

```
div{

width: 100px;  height: 100px;

transform: translate(10px, 50px); //сдвигает элемент по осям x и y на 10px и 50px по отношению к своему первоначальному положению.

}
```



scale(x,y)- масштаб

`scale(x,y)` Масштабирует элементы, делая их больше или меньше. Значения от 0 до 1 уменьшают элемент. Первое значение масштабирует элемент по ширине, второе — по высоте. Отрицательные значения отображают элемент зеркально.

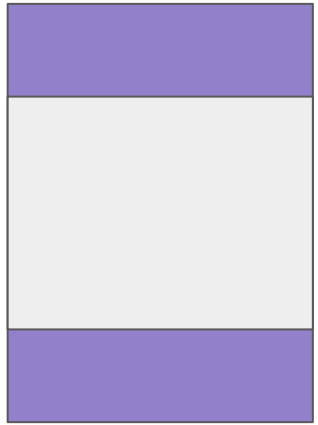
`scaleX(n)` Функция масштабирует элемент по ширине, делая его шире или уже. Если значение больше единицы, элемент становится шире, если значение находится между единицей и нулем, элемент становится уже. Отрицательные значения отображают элемент зеркально по горизонтали.

`scaleY(n)` Функция масштабирует элемент по высоте, делая его выше или ниже. Если значение больше единицы, элемент становится выше, если значение находится между единицей и нулем — ниже. Отрицательные значения отображают элемент зеркально по вертикали.

```
div:hover {
      transform: scale(0.8);
  }
```



```
div:hover {
      transform: scaleY(2);
  }
```



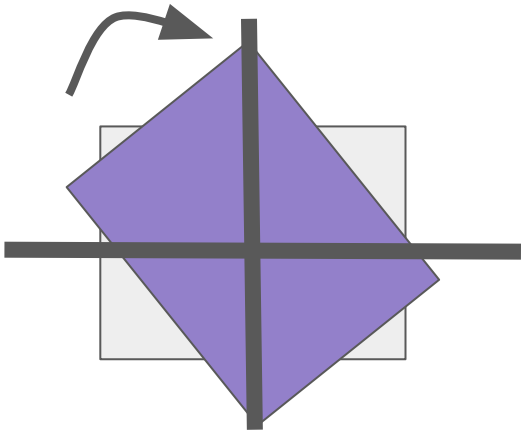
rotate(угол) - поворот

`rotate(угол)` Поворачивает элементы на заданное количество градусов (**`deg`**), вокруг исходного центра(определенного с помощью **`transform-origin`**).

При положительном значении — элемент поворачивается по часовой стрелке.
Отрицательные значения от `-1deg` до `-360deg` поворачивают элемент против часовой стрелки

Значение `rotate(360deg)` поворачивает элемент на 1 полный оборот.
Значение `rotate(720deg)` поворачивает элемент на два полных оборота.

```
div:hover {
    transform: rotate(45deg);
}
```



skew(x,y) - наклон элемента

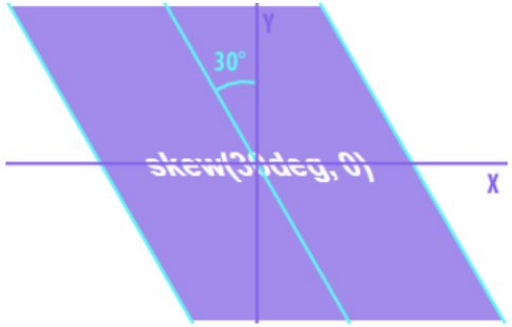
`skew(x-угол,y-угол)` Используется для деформирования (искажения) сторон элемента относительно координатных осей.

Координаты каждой точки изменяются на величину, пропорциональную указанному углу и расстоянию до точки, относительно которой выполняется трансформация (origin); таким образом, чем больше расстояние до смещаемой точки элемента, тем больше она будет смещена по горизонтали или вертикали.

Значения указываются в углах, градусах, радианах и оборотах.

`skewX(угол)` Задаёт угол наклона по горизонтали, то есть отвечает за наклон вертикальных линий к оси X. ***Движение происходит от верхней части элемента и против часовой стрелки.***

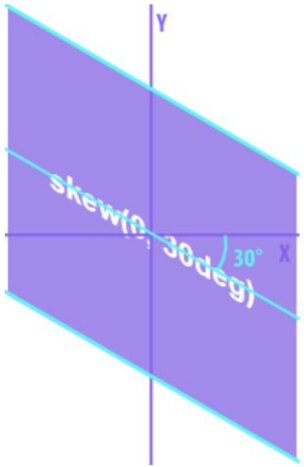
Положительное значение наклоняет влево, отрицательное вправо.



`skewY(угол)` Задаёт угол наклона по вертикали, то есть отвечает за наклон горизонтальных линий к оси Y. ***Направление движения — по часовой стрелке, начиная от правой стороны элемента***

Положительное значение наклоняет вниз, отрицательное вверх.

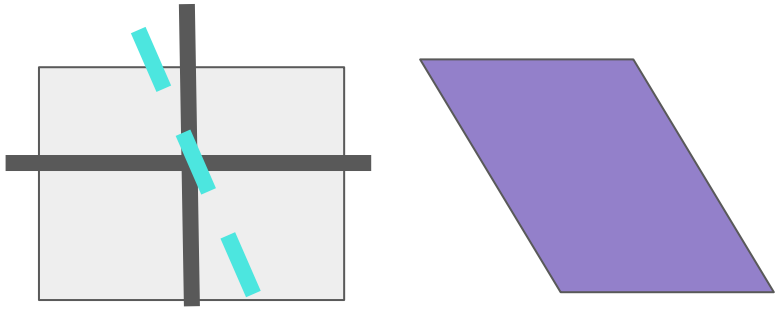
Если значение не указано, то по умолчанию принимается равным 0 и искажения по вертикали не происходит.



наклон по горизонтали

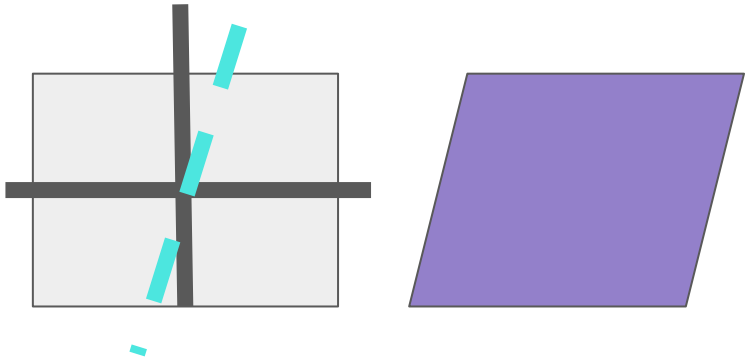
наклон влево

```
div:hover {
// наклон по горизонтали на 40 градусов
    transform: skewX(40deg);
   }
```



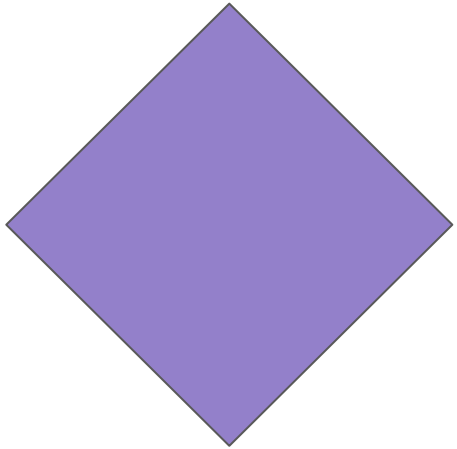
наклон вправо

```
div:hover {
// наклон по горизонтали на -20 градусов
    transform: skewX(-20deg);
   }
```



```
div{
        width:200px;
        height:100px;
        display: block;
        background: #f00;
        transform:  skewX(-40deg) rotate(25deg);
        }
```





`transform: matrix(a, b, c, d, x, y)` - объединяет функции трансформации

`none` Значение по умолчанию, означает отсутствие трансформации. Также отменяет трансформацию для элемента из группы трансформируемых элементов.

`matrix(a, b, c, d, x, y)` Смещает элементы и задает способ их трансформации, позволяя объединить несколько функций 2D-трансформаций в одной. В качестве трансформации допустимы поворот, масштабирование, наклон и изменение положения.

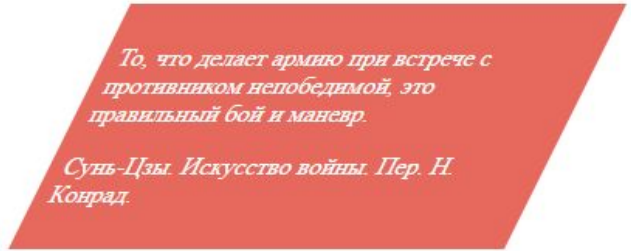
- Значение **`a`** изменяет **`масштаб по горизонтали`**. Значение от 0 до 1 уменьшает элемент, больше 1 — увеличивает.
- Значение **`b`** деформирует (**`сдвигает) стороны элемента по оси Y`**, положительное значение — вверх, отрицательное — вниз.
- Значение **`c`** деформирует **`(сдвигает) стороны элемента по оси X`**, положительное значение — влево, отрицательное — вправо.
- Значение **`d`** изменяет **`масштаб по вертикали`**. Значение меньше 1 уменьшает элемент, больше 1 — увеличивает.
- Значение **`x`** **`смещает элемент по оси X`**, положительное — вправо, отрицательное — влево.
- Значение **`y`** **`смещает элемент по оси Y`**, положительное значение — вниз, отрицательное — вверх..

```
/* Трансформация масштаба*/
transform: matrix(1.2, 0, 0, 1, 0, 0);
```

То, что делает армию при встрече с противником непобедимой, это правильный бой и маневр.

Сунь-Цзы. Искусство войны. Пер. Н. Конрад.

```
/* Трансформация наклона*/
transform: matrix(1, 0, -0.5, 1, 0, 0);
```



transform-origin - точка трансформации

Свойство позволяет **сместить центр трансформации**, относительно которого происходит изменение положения/размера/формы элемента. Значение по умолчанию — `center`, или `50% 50%`. Задается только для трансформированных элементов.

```
transform-origin: <x> <y>
```

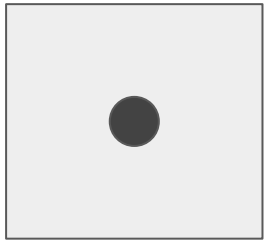
<x> Координата по оси X. Может принимать следующие значения: <размер> | <проценты> | left | center | right <размер> — любая единица измерения CSS.

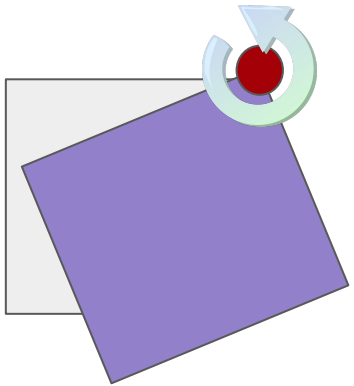
<y> Координата по оси Y. Может принимать следующие значения: <размер> | <проценты> | top | center | bottom

```
div {
    background: #fc0;
    padding: 10px;
    display: inline-block;
    border: 1px solid #000;
    transition: 1s;
    /* Точка поворота в правом верхнем углу */
    transform-origin: 100% 0;
   }

div:hover {
      /* Поворачиваем на 20 градусов против часовой
      стрелки относительно правого верхнего угла. */
    transform: rotate(-20deg);

   }
```





Google Trans

Трансформация

1

2 Lorem ipsum dolor sit.

3

При наведении на элементы



в обычном состоянии



Каркас блока #1

Иконка или кнопки от шрифта Font Awesome

при наведении на блок



Каркас блока #1

Иконка или кнопки от шрифта Font Awesome

```
<div class="block">
  <div class="marker">
        <i class="fa fa-globe"></i>
  </div>
  <h3>Каркас блока #1</h3>
  <p>Иконка от шрифта Font Awesome</p>
</div>
```

3D-трансформации элементов

Модуль 3D Transforms расширяет спецификацию CSS 2D Transforms, позволяя преобразовывать элементы в трехмерном пространстве. Новые функции преобразования для свойства `transform` выполняют трехмерные преобразования, расширяя координатное пространство до трех измерений, добавляя ось Z, перпендикулярную плоскости экрана, которая увеличивается по направлению к зрителю

transform-style: flat | preserve-3d

Свойство `transform-style` позволяет преобразованным 3D-элементам и их 3D-потомкам использовать общее трехмерное пространство, выстраивая иерархии трехмерных объектов. Отображение 3D-потомков определяется моделью — так называемым **контекстом 3D-рендеринга**. Отображение зависит от z-позиции элементов в трехмерном пространстве, и если 3D-преобразования этих элементов вызывают пересечение, то они отображаются с пересечением.

Свойство устанавливается для родительского элемента.

flat - Дочерние элементы лежат в плоскости экрана.

preserve-3d - Дочерние элементы будут отображаться в 3D-пространстве. Получается, что при *transform-style: preserve-3d к*аждый дочерний элемент получает ***независимую*** от родителя плоскость, к которой можно применять 3D-трансформации.

https://doka.guide/css/transform-style/

perspective: none | <размер>

В нормальном потоке элементы отображаются плоскими и в той же плоскости, что и блок, содержащий их. Двумерные функции преобразования могут изменять внешний вид элемента, но этот элемент по-прежнему отображается в той же плоскости, что и содержащий его блок.

Свойства `perspective` и `perspective-origin` можно использовать для добавления ощущения глубины в сцену, делая элементы выше по оси Z (ближе к зрителю) и кажущимися большими, а те, которые находятся дальше — меньшими.

Свойство `perspective` активирует 3D-пространство внутри элемента, содержащего дочерние трансформированные элементы и применяется к ним.

Установка 3D-перспективы perspective

perspective: 400px;



perspective: 800px;



Чем больше значение функции перспективы, тем дальше от смотрящего расположен элемент. Значение должно быть больше нуля.

perspective-origin: <позиция>

Обычно предполагаемое положение глаза зрителя находится в центре рисунка. Свойство `perspective-origin` управляет точкой начала координат, позволяя изменять направление трансформации дочернего 3D-элемента. Свойство должно использоваться вместе со свойством `perspective` для **родительского элемента** и свойством `transform` для дочернего элемента.

backface-visibility: visible | hidden

Определяет, показывать обратную сторону элемента или нет. Как правило, обратная сторона становится видна при трансформации элемента, например, при его вращении относительно горизонтальной или вертикальной оси в трёхмерном пространстве.

visible - Обратная сторона элемента видна и отображается через переднюю поверхность элемента зеркально.

hidden - Обратная сторона не видна, скрываясь за передней поверхностью элемента.

примеры

https://designmodo.com/demo/backface-visibility/coffee.html

Функции 3D-трансформации

```
transform: translate3d(100px, 100px, -200px);

transform: scale3d(2, 1, 3);

transform: rotate3d(1, 1, 1, 45deg);
```

Вращение в 3D

`rotate3d` задает поворот элемента на заданный угол одновременно относительно оси X, Y, Z в трехмерном пространстве.

transform: rotate3d(X, Y, Z, угол);

В качестве значений функция принимает 4 параметра, записанные через запятые. Первые три параметра определяют вектор оси в трехмерном пространстве, относительного которой будет осуществляться поворот. Четвертый параметр задает угол поворота.

transform-style: preserve-3d;

Дверь 3D

```
<div class="trans_rotate">
      <p></p>
</div>
```

```
<style>
      .trans_rotate{
              width:200px; height: 200px;
              background: #000;
             perspective: 800px ;}

      .trans_rotate p{
                width:100%; height: 100%;
                background: aqua;
                transform-style: preserve-3d;
      }

       .trans_rotate:hover p{
                /*поворачивает на 180 градусов по Y */
                transform:  rotate3d(0,1,0,180deg);
                background: pink;

      }

</style>
```

Вращение карточки



Кроссовки Adidas

«Бренд с тремя полосками» является одной из крупнейших спортивных компаний в истории

https://sneakers-type.ru/wp-content/uploads/2021/08/xKrossovki-Adidas-Niteball-Black-Green-e1629654575958.jpg.pagespeed.ic.qin9G7SRuF.webp

Кроссовки Adidas Niteball Black Green
«Бренд с тремя полосками» является одной из крупнейших спортивных компаний в истории

CSS3-фильтры -воспроизводят в браузере визуальные эффекты, похожие на фильтры Photoshop. Фильтры можно добавлять не только к изображениям, но и к любым непустым элементам.

`blur()` Значение задается в единицах длины, например `px`, `em`. Применяет размытие по Гауссу к исходному изображению. Чем больше значение радиуса, тем больше размытие. Если значение радиуса не задано, по умолчанию берется `0`.

`brightness()` Значение задается в `%` или в десятичных дробях. Изменяет яркость изображения. Чем больше значение, тем ярче изображение. Значение по умолчанию `1`.

`contrast()` Значение задается в `%` или в десятичных дробях. Регулирует контрастность изображения, т.е. разницу между самыми темными и самыми светлыми участками изображения/фона. Значение по умолчанию `100%`. Нулевое значение скроет исходное изображение под темно-серым фоном. Значения, увеличивающиеся от `0` до `100%` или от `0` до `1`, будут постепенно открывать исходное изображение до оригинального отображения, а значения свыше будут увеличивать контраст между светлыми и темными участками.

`grayscale()`- убирает цвета, делает черно-белыми/серыми.

`opacity()` Фильтр работает аналогично со свойством `opacity`, добавляя прозрачность элементу. Отличительная особенность — браузеры обеспечивают аппаратное ускорение для фильтра, что позволяет повысить производительность. Дополнительный бонус — фильтр можно одновременно сочетать с другими фильтрами, создавая при этом интересные эффекты. Значение задается только в `%`, `0%` делает элемент полностью прозрачным, а `100%` не оказывает никакого эффекта.

`saturate()` Управляет насыщенностью цветов, работая по принципу контрастного фильтра. Значение `0%` убирает цветность, а `100%` не оказывает никакого эффекта. Значения от `0%` до `100%` уменьшают насыщенность цвета, выше `100%` — увеличивают насыщенность цвета. Значение может задаваться как в `%`, так и целым числом, `1` эквивалентно `100%`.

CSS3-фильтры. Примеры

```
filter: blur(3px);
filter: brightness(50%);
filter: brightness(.5);
filter: contrast(20%);
filter: contrast(.2);
filter: drop-shadow(2px 3px 5px black);
filter: grayscale(.5);
filter: grayscale(50%);
filter: hue-rotate(180deg);
filter: invert(100%);
filter: opacity(30%);
filter: saturate(300%);
```

CSS3-анимация

создание анимации базируется на ключевых кадрах, которые позволяют автоматически воспроизводить и повторять эффекты на протяжении заданного времени, а также останавливать анимацию внутри цикла.

CSS3-анимация может применяться практически для всех html-элементов, а также для псевдоэлементов `:before` и `:after`.

```
animation: animation-name animation-duration animation-timing-function
animation-delay animation-iteration-count animation-direction;
```

- animation-name: название анимации;
- animation-duration: как долго длится анимация;
- animation-timing-function: как вычисляются промежуточные состояния;
- animation-delay: анимация начинается спустя некоторое время;
- animation-iteration-count: сколько раз должна выполняться анимация(infinite - все время);
- animation-direction: должно движение идти в обратную сторону или нет(normal,reverse,alternate);
- animation-fill-mode: какие стили применяются до начала анимации и после её завершения.
- animation-play-state: определяет, будет ли анимация запущена или приостановлена(`running,paused`.

https://html5book.ru/css3-animation/

Ключевые кадры @keyframes

@keyframes устанавливает ключевые кадры при анимации элемента. Ключевой кадр это свойства элемента (прозрачность, цвет, положение и др.), которые должны применяться к элементу в заданный момент времени. Таким образом, анимация представляет собой плавный переход стилевых свойств от одного ключевого кадра к другому. Ключевые кадры определяют поведение одного цикла анимации; анимация может повторяться ноль или более раз.

Ключевые кадры указываются с помощью правила `@keyframes`, определяемого следующим образом:

```
@keyframes имя анимации { список правил }
```

```
@keyframes box {
  from { left: 0; }
  50% { left: 50px; }
  to { left: 300px; }
}
```

Ключевые кадры создаются с помощью ключевых слов `from` и `to` (эквивалентны значениям `0%` и `100%`) или с помощью процентных пунктов, которых можно задавать сколько угодно.

Пример использования

```
.loading-button {
 animation: bouncing 0.5s linear 0s alternate both infinite;
 }

@keyframes bouncing{
  0%  { bottom: 0; box-shadow: 0 0 5px rgba(0,0,0,0.5); }
  100% { bottom: 50px; box-shadow: 0 50px 50px rgba(0,0,0,0.1); }
}
```

примеры

https://htmlacademy.ru/blog/html/animations

Задание

Реализовать html-страницу с прогресс-баром.

При открытии страницы этот прогресс-бар должен двигаться слева направо бесконечно.

Пример конечного результата:

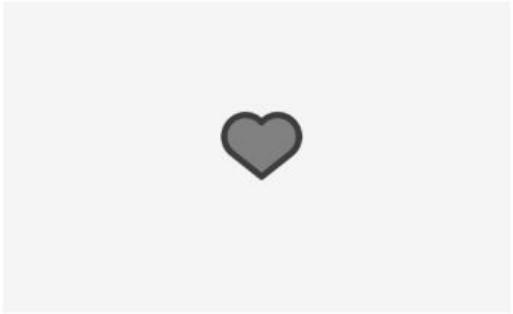


Задание

Реализовать html-страницу с живой иконкой.

При загрузке страницы отображается черно-белая иконка. Если навести на нее мышкой, то она медленно (5 секунд) увеличивается и становится цветной. Если мышку увести, то иконка резко возвращается к первоначальному состоянию.

Пример первоначального состояния:



Пример состояния иконки при наведении мышкой, когда анимация закончилась:




СВЯЗЬ

Задание

Задание 3

Реализовать html-страницу со всплывающими иконками.

Требования:

- текст **Hello, my site!** отображается на странице сразу (анимация не нужна);
- иконки выезжают снизу на свои места, каждая за 2 секунды;
- иконки выезжают друг за другом с разницей в 0.5 секунды;
- иконки начинают выезжать при загрузке страницы и после окончания анимации остаются на своих местах.

Пример конечного результата:

